

TP 1 Big Data

Rappel ~ Linux, Docker

Préparation de l'environnement Hadoop

Objectif :

Ce premier TP a pour objectif de préparer l'environnement de travail qui sera utilisé pendant les prochains TPs du module.

À la fin de ce TP, vous devez être capables de :

- manipuler des fichiers et répertoires sous Linux ;
- utiliser des commandes Unix pour rechercher et filtrer des données ;
- construire des pipelines avec | ;
- comprendre les notions d'**image Docker** et de **conteneur** ;
- lancer, arrêter et inspecter un conteneur ;
- utiliser quelques commandes Docker essentielles ;
- comprendre le rôle des volumes Docker ;
- utiliser Docker Compose ;
- télécharger et démarrer l'environnement Hadoop (préparation du TP2).

Partie A — Rappel Linux

A.1. Navigation

Tester les commandes suivantes et préciser le rôle de chacune d'elles.

```
pwd
ls
ls -l
ls -la
mkdir -p travail/data travail/logs travail/resultats
cd travail
pwd
ls
cd ..
```

A.2. Manipulation des fichiers

Tester les commandes suivantes :

```
touch travail/data/test.txt
cp travail/data/test.txt travail/resultats/test-copy.txt
mv travail/resultats/test-copy.txt travail/resultats/test-final.txt
rm travail/resultats/test-final.txt
```

- Quelle est la différence entre `cp` et `mv` ? Pourquoi faut-il être prudent avec `rm` ?

A.2. Consultation

Tester les commandes suivantes :

```
printf "ligne 1\nligne 2\nligne 3\nligne 4\nligne 5\n" > travail/data/test.txt
cat travail/data/test.txt
head -n 2 travail/data/test.txt
tail -n 2 travail/data/test.txt
less travail/data/test.txt
```

- Dans un contexte Big Data, pourquoi éviter `cat` pour consulter un fichier volumineux ? quelles commandes utiliser à la place ?

A.3. Linux et traitement de logs

Le fichier `data/access.log` contient des requêtes HTTP. Tester les commandes suivantes et préciser le rôle de chacune d'elles.

```
cat data/access.log
wc -l data/access.log
grep "404" data/access.log
grep "500" data/access.log
grep "404" data/access.log | wc -l
cut -d' ' -f1 data/access.log
cut -d' ' -f1 data/access.log | sort | uniq -c
cut -d' ' -f1 data/access.log | sort | uniq -c | sort -nr
grep "404" data/access.log | cut -d' ' -f1 | sort | uniq -c | sort -nr
```

- Que permet d'obtenir le dernier pipeline ?
- Pourquoi un pipeline Linux classique atteint-il ses limites lorsque les données atteignent plusieurs centaines de Go ou plusieurs To ?

Partie B — Découverte de Docker

B.1. Vérification

Soient les deux commandes suivantes :

```
docker --version
docker compose version
```

Quelle différence entre Docker et Docker Compose ?

B.2. Première image et premier conteneur

Tester les commandes suivantes :

```
docker run hello-world
docker images
docker ps
docker ps -a
```

- Quelle est la différence entre une image Docker et un conteneur Docker ?
- Pourquoi `hello-world` apparaît-il dans `docker ps -a` mais pas nécessairement dans `docker ps` ?

B.3. Conteneur interactif

Exécuter la commande suivante :

```
docker run -it ubuntu bash
```

Dans le conteneur :

```
pwd
ls
cat /etc/os-release
exit
```

Le système de fichiers du conteneur est-il identique à celui de la machine hôte ? Justifiez votre réponse à partir des commandes exécutées.

B.4. Conteneur en arrière-plan

Exécuter les commandes suivantes :

```
docker run -d --name linux-test ubuntu sleep 3600
docker ps
docker exec -it linux-test bash
```

Dans le conteneur :

```
hostname
ls
exit
```

Puis:

```
docker inspect linux-test
docker logs linux-test
docker stop linux-test
docker rm linux-test
```

- Quelle est la différence entre `docker run` et `docker exec` lorsqu'on souhaite accéder à un terminal dans un conteneur ?
- À quoi servent `docker ps`, `docker logs` et `docker inspect` pour diagnostiquer un problème ?

B.4. Volumes et Docker Compose

Exécuter et analyser les commandes suivantes :

```
mkdir -p ~/bigdata-tp/shared
echo "Bonjour depuis la machine hôte" > ~/bigdata-tp/shared/message.txt
docker run --rm -v ~/bigdata-tp/shared:/data ubuntu cat /data/message.txt
```

B.5. Docker Compose

Créer `compose-test.yaml` :

```
services:
  linux1:
    image: ubuntu
    command: sleep 3600

  linux2:
    image: ubuntu
    command: sleep 3600
```

Puis, exécuter les commandes suivantes :

```
docker compose -f compose-test.yaml up -d
docker compose -f compose-test.yaml ps
docker compose -f compose-test.yaml exec linux1 bash
docker compose -f compose-test.yaml down
```

Remarque : `-d` signifie **detached** (« en arrière-plan »).

- Quel est l'intérêt de Docker Compose par rapport à plusieurs `docker run` ?
- Pourquoi notre futur environnement Hadoop aura-t-il besoin de plusieurs conteneurs ?

Partie C — Préparation de l'environnement Hadoop

L'environnement contiendra : Un Namenode, Deux Datanodes : Datanode1 et Datanode2

C.1. Télécharger l'image

Depuis tp1, exécuter :

```
docker compose pull
docker images
```

Remarque : Lors de la première utilisation, Docker télécharge l'image Hadoop depuis un registre Docker si elle n'est pas déjà présente. Une fois téléchargée, elle reste stockée localement.

C.2. Démarrer le cluster

```
docker compose up -d
docker compose ps
```

Vous devez retrouver namenode, datanode1 et datanode2.

C.3. Consulter les logs

```
docker compose logs namenode
docker compose logs datanode1
```

- Pourquoi les logs sont-ils importants lorsqu'un service Big Data ne démarre pas correctement ?

C.4. Entrer dans le NameNode

```
docker compose exec namenode bash
java -version
hdfs version
```

C.5. Première commande HDFS

Toujours dans le NameNode :

```
hdfs dfs -ls /
```

Comparer avec :

```
ls /
```

C.6. Interface Web

Depuis le navigateur : <http://localhost:9870>

Observer les informations sur les DataNodes, la capacité HDFS et l'état du système de fichiers.